

Implementación y Visualización de Algoritmos de Búsqueda

BFS · DFS · UCS · A sobre grafos y laberintos*

1. Objetivo

Implementar y comparar los algoritmos de búsqueda no informada (BFS, DFS, UCS) e informada (A*) sobre dos escenarios: grafos/árboles y laberintos representados como matrices. Se busca observar las diferencias en costo de la solución, cantidad de nodos visitados y calidad del camino encontrado.

2. Arquitectura y decisiones de diseño

La decisión central fue definir una interfaz de problema única (clase Problema) que expone cuatro operaciones: estado inicial, prueba de objetivo, generación de vecinos con costo, y heurística. Gracias a esto, los cuatro algoritmos se implementan UNA sola vez y funcionan sin cambios tanto para grafos como para laberintos. Cada escenario es solo una implementación distinta de la misma interfaz.

- Grafo (ProblemaGrafo): admite lista de adyacencia, lectura de CSV como lista de aristas (origen,destino, costo) o como matriz de adyacencia, y coordenadas opcionales para la heurística euclidiana.
- Laberinto (ProblemaLaberinto): matriz de 0 (libre) y 1 (barrera). El estado es (fila, columna); el movimiento es en 4 direcciones con costo 1, con opción de 8 direcciones. La heurística es la distancia Manhattan al objetivo.
- Visualización (matplotlib): resalta barreras, celdas visitadas, camino final, inicio y objetivo; para grafos se usa networkx con el camino y los nodos visitados destacados.

Detalles de implementación relevantes: BFS usa una cola FIFO; DFS una pila LIFO; UCS y A* usan heapq con un contador incremental que desempata y evita comparar estados directamente. UCS ordena por $g(n)$ (costo acumulado) y A* por $f(n) = g(n) + h(n)$. Tanto UCS como A* permiten reabrir un nodo si se descubre un camino más barato hacia él.

3. Resultados: laberinto

Laberinto de 10x10 con inicio en (0,0) y objetivo en (9,9). Los cuatro algoritmos encuentran un camino de costo 18, pero difieren en cuántos nodos expanden:

Algoritmo	Nodos visitados	Costo / pasos	Frontera max
BFS	43	18	4
DFS	43	18	5
UCS	43	18	4
A*	33	18	4

Comparacion de algoritmos sobre laberinto

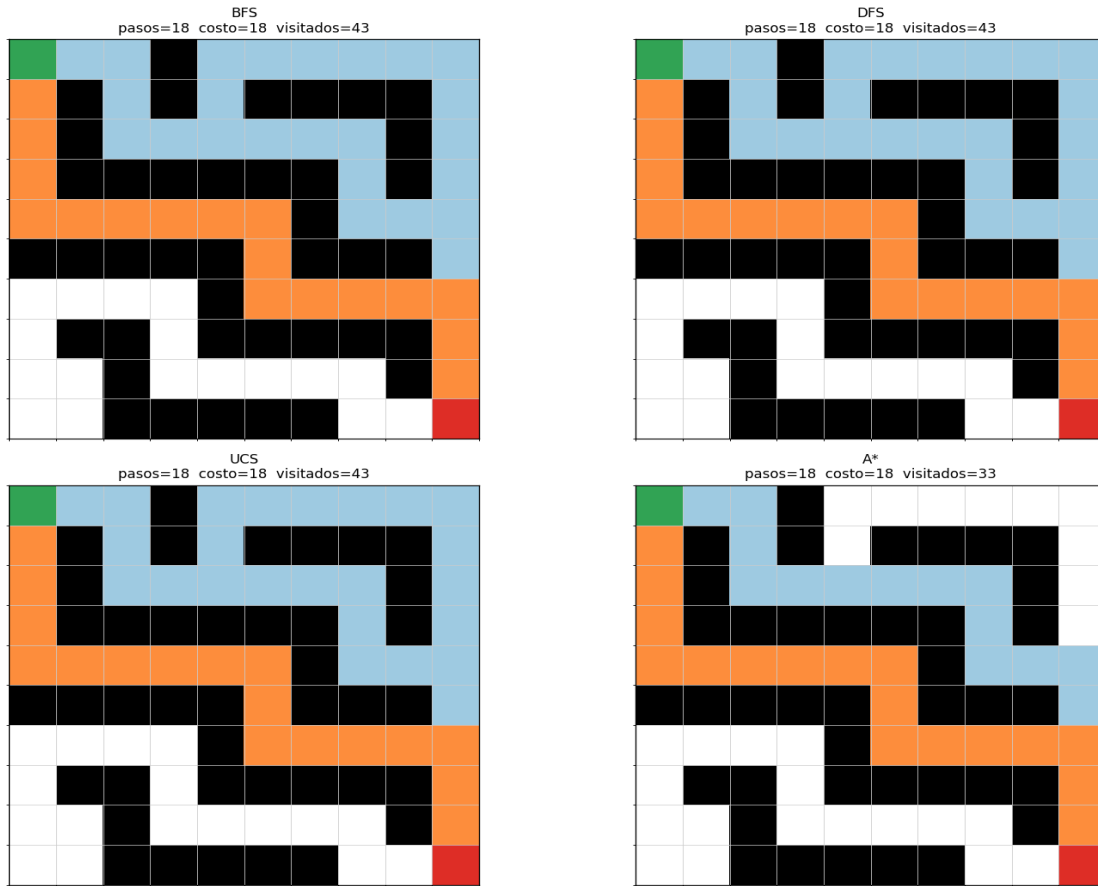


Figura 1. Recorrido de cada algoritmo. Verde=inicio, rojo=objetivo, naranja=camino, celeste=visitado, negro=barrera.

Observación clave: A* expande solo 33 celdas frente a 43 de BFS y UCS, porque la distancia Manhattan lo guía hacia el objetivo y evita explorar zonas alejadas. En la figura, A* deja más celdas blancas (no visitadas) en la esquina inferior. En este laberinto de costo uniforme, BFS y UCS recorren prácticamente lo mismo, ya que con costos iguales UCS degenera en BFS.

4. Resultados: grafo

Grafo ponderado de 7 nodos (A..G) con inicio A y objetivo G. Aquí las diferencias entre búsqueda no informada por niveles y búsqueda por costo se hacen evidentes:

Algoritmo	Visitados	Costo	Pasos
BFS	7	9	3
DFS	7	9	3
UCS	7	7	5
A*	6	7	5

Comparación de algoritmos sobre grafo

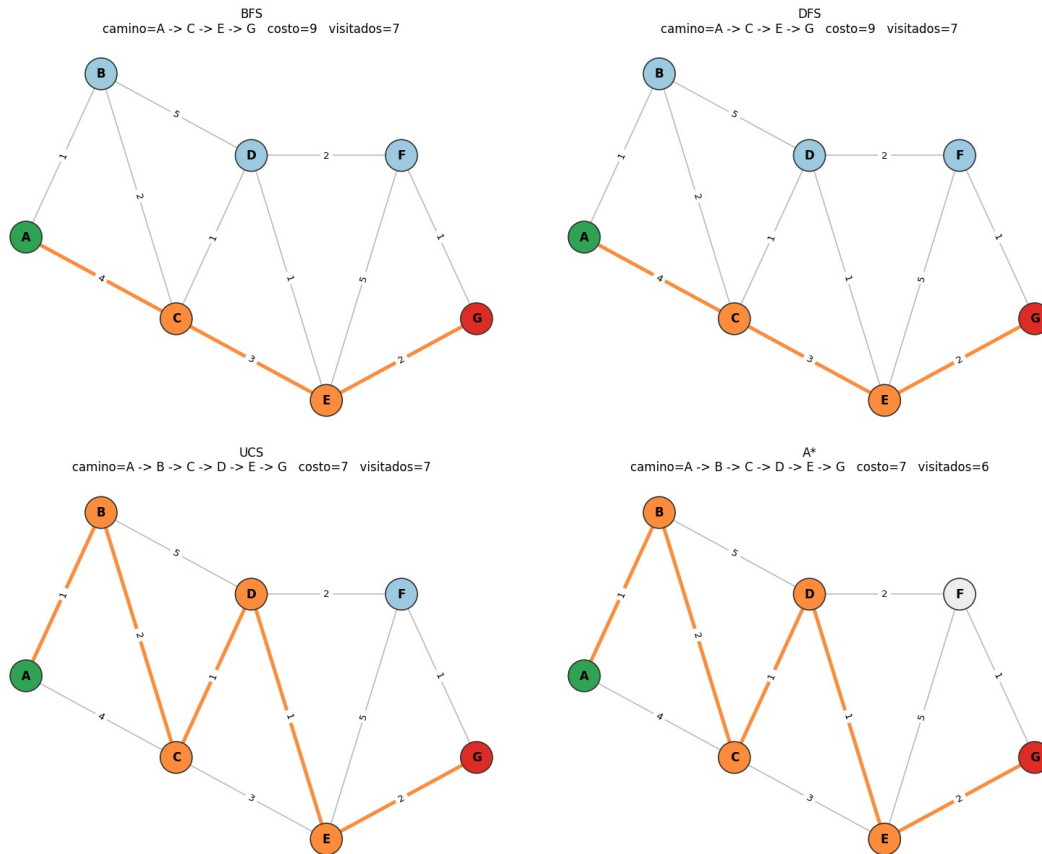


Figura 2. BFS/DFS hallan A-C-E-G (costo 9, 3 saltos); UCS/A* hallan A-B-C-D-E-G (costo 7, 5 saltos).

Observación clave: BFS encuentra el camino con MENOS saltos (A-C-E-G, 3 aristas) pero NO el más barato, porque ignora el peso de las aristas. UCS y A* encuentran el camino de MENOR COSTO (A-B-C-D-E-G, costo 7) aunque tenga más saltos. A* además expande un nodo menos que UCS (no necesita visitar F) gracias a la heurística. Esto ilustra que 'más corto en pasos' y 'más barato en costo' no son lo mismo.

5. Comparación general

- **BFS:** completo y óptimo en número de pasos cuando todos los costos son iguales. Ignora pesos, por lo que en grafos ponderados puede dar caminos caros. Memoria alta (guarda toda la frontera de un nivel).
- **DFS:** bajo uso de memoria, pero ni óptimo ni necesariamente corto; puede tomar caminos muy largos. Útil cuando cualquier solución sirve o para explorar a fondo.
- **UCS:** óptimo en costo con pesos no negativos (es Dijkstra con objetivo). Sin guía hacia la meta, expande nodos en todas direcciones.
- **A*:** óptimo en costo si la heurística es admisible (Manhattan lo es para 4 direcciones con costo unitario). Combina el costo real $g(n)$ con la estimación $h(n)$, por lo que expande menos nodos que UCS dirigiendo la búsqueda hacia el objetivo.

6. Conclusiones

La elección del algoritmo depende del problema. Si todas las acciones cuestan lo mismo y se busca el camino más corto en pasos, BFS basta. Si hay costos distintos, UCS garantiza el óptimo, y A* lo logra explorando menos cuando se dispone de una heurística admisible, siendo la mejor opción para laberintos grandes. DFS queda reservado para cuando importa la memoria o basta cualquier solución. Los experimentos confirman estos comportamientos: A* redujo los nodos visitados (33 vs 43 en el laberinto, 6 vs 7 en el grafo) manteniendo el costo óptimo, y se evidenció la diferencia entre minimizar pasos (BFS) y minimizar costo (UCS/A*).

7. Cómo ejecutar

Comparar los 4 algoritmos y generar las figuras:

```
python main.py --comparar laberinto
```

```
python main.py --comparar grafo
```

Ejecutar un algoritmo concreto:

```
python main.py --escenario laberinto --algoritmo A* --inicio 0,0 --objetivo 9,9
```

```
python main.py --escenario grafo --algoritmo UCS --inicio A --objetivo G
```

Modo interactivo y pruebas automáticas:

```
python main.py | python pruebas.py
```